

weekly-lab 作業流程

每週練習台 · 操作框架與除錯手冊 | repo: sh-marketlab/weekly-lab | 站台: sh-marketlab.github.io/weekly-lab

核心心智模型：三個地方，各管一件事。
PowerShell 只負責「改程式碼」。 **GitHub 網頁** 只負責「改資料檔」。 **Actions** 是唯一會真的去抓資料的地方——在網頁上 commit 只會重新發佈網站，不會重新抓資料，所以改完資料一定要手動 Run workflow。

A · 每週例行節奏

時間（台灣）	誰做	做什麼
週五 07:00	自動	排程跑 <code>thu</code> ，抓美股週四收盤 → Step 1 水管 + Step 2 板塊
週五 白天	你	看儀表板。順手更新主權 CDS（每月初再加 PMI）→ 見 流程 B
週六 07:00	自動	排程跑 <code>fri</code> ，抓美股週五收盤（完整週）→ Step 3 主題輪動
週末	你	推演。在「04 · 預測與批改」寫下週預測 → 見 流程 C
週二 07:00	自動	排程跑 <code>mon</code> ，抓美股週一收盤，自動批改前六項
週二 白天	你	看 Step 5 批改結果，標記誤差類型 → 見 流程 C 後半

手動 Run workflow 時，stage 一定要自己選對。 規則是「你想看哪一天的美股收盤就選哪一天」，而且那天必須已經收盤。
台灣週五早上想看最新資料 → 選 `thu`（美股週五那場要台灣週六凌晨才收）。選錯 `fri` 會拿到上一週的週五，數字看起來正常但整整晚一週。

B · 更新手動資料（主權 CDS、PMI、MOVE 備援）

- 1

填值
「05 · 設定與手填」→ 該列填
日期
+
數值
→ 按「加入」。日期填資料本身的日期，要對齊分頁上方 meta 列顯示的「本期」日期。PMI 是月度資料，用當月 1 號。

儀表板
- 2

產生 JSON → 複製
產出的是「repo 既有資料 + 你新填的」完整內容，不是只有新的那幾筆。

儀表板
- 3

貼回 data/manual_input.json
按「在 GitHub 編輯此檔 ↗」→
Ctrl+A 全選再貼上
（整份取代，不是接在後面）→ Commit changes。

GitHub
- 4

Run workflow
Actions → 左欄 `weekly-lab` → Run workflow → 選對 stage → 綠色 Run workflow。等綠勾（約 2–4 分鐘）。

GitHub
- 5

Ctrl + Shift + R
強制刷新。不強制刷新會看到快取的舊頁面，誤以為沒更新。

瀏覽器

沒貼回 repo 就等於沒填。 你填的值先存在瀏覽器 localStorage（讓你立刻看到），但排程是在 GitHub 的機器上跑、讀的是 repo 裡的檔案。頁面上「本機草稿 N 筆，尚未貼回 repo」就是在提醒這件事，貼回去後那行會消失。

本機那份 manual_input.json 會落後。 你是在 GitHub 網頁改的，電腦裡還是舊的。下次要在 PowerShell 動任何東西前，先 `git pull --rebase`。

C • Step 4 寫預測 / Step 5 批改

週末：寫下週預測

1 填八個欄位

[儀表板](#)

「04 • 預測與批改」→ 每欄選判斷 + 給 1-5 分信心 + 寫理由

◦ 理由欄位比選項重要——週一要驗的是「理由對不對」，不是「結果對不對」。最後填「大魔王意圖推論」。

2 產生 JSON → 複製

[儀表板](#)

3 貼回 data/predictions.json

[GitHub](#)

Ctrl+A 全選再貼上 → Commit changes。

預測不用手動 Run workflow。這是跟流程 B 唯一不同的地方——週二早上的排程會自己去讀 data/predictions.json、抓週一收盤、自動判定並把結果寫回去。你只要確保週二之前有貼回 repo 就好。沒貼的話，週二那趟只會印「沒有待批改的預測」然後跳過。

週二：批改與標記

1 看 Step 5 自動判定結果

[儀表板](#)

前六項（整體方向、早盤手法、資金流向、龍頭表現、市場廣度、波動度）機器已判好，紅字綠字沒有商量餘地。後兩項（背離是否兌現、大魔王意圖）機器判不了，自己標。

2 標記誤差類型

[儀表板](#)

判錯的要選一個類型，並寫「應該要怎麼判斷才對」。

答對的題目也可以標

：結果對但過程錯 → 選「對但理由錯（運氣）」；推論健全但被外生事件打斷 → 選「理由對但結果錯」。

3 產生 JSON → 貼回 predictions.json → Commit

[GitHub](#)

累積統計要看什麼

- 命中率只是入場券。真正有訊息量的是下面兩個。
- 信心校準：標 5 分把握時實際命中率是不是接近 100%？負落差＝過度自信。樣本少於 10 的那一列先別當真。
- 誤差類型分布：如果同一個類型一直排第一，那不是運氣問題，是框架缺了一層。

D • 更新程式碼

1 下載檔案 → 自動歸位

[PowerShell](#)

解壓縮到下載資料夾任何位置皆可，指令的 -Recurse 會遞迴尋找。在 weekly-lab 資料夾 Shift+右鍵 → 在終端中開啟。

```
$dl = "$env:USERPROFILE\Downloads"
function Grab($name, $dest) {
    $f = Get-ChildItem $dl -Recurse -Filter $name -File -ErrorAction SilentlyContinue |
        Sort-Object LastWriteTime -Descending | Select-Object -First 1
    if ($f) { Copy-Item $f.FullName $dest -Force; Write-Host "OK   $name" -ForegroundColor Green }
    else    { Write-Host "找不到 $name" -ForegroundColor Red }
}
Grab "build.py"      "src\build.py"
Grab "index.html"    "docs\index.html"
Grab "sectors.yaml"  "config\sectors.yaml"
```

2 確認真的換到新版

[PowerShell](#)

```
git status --short # 應看到 M 開頭的檔案清單
```

顯示 nothing to commit 就代表步驟 1 沒真的覆蓋到，別往下走。

3 推上去（順序不能錯）

PowerShell

```
git add .
git commit -m "說明這次改了什麼"
git pull --rebase
git push
```

一次貼一行

。整段貼上時 PowerShell 不管前一行有沒有失敗會照跑，錯誤會被沖過去。

4 Run workflow

GitHub

Actions → weekly-lab → Run workflow → 選對 stage → Run。等綠勾。

5 Ctrl + Shift + R

瀏覽器

順序必須是 **add → commit → pull --rebase → push**。pull --rebase 要把你的 commit 暫時搬開、拉遠端的、再疊回去，所以工作區必須乾淨。還沒 commit 就 pull，會被擋下 cannot pull with rebase: You have unstaged changes。

為什麼幾乎每次都要 **pull**。這個 repo 每週有三次機器人自動 commit（週五、週六、週二早上），你本機必然落後。--rebase 讓歷史維持一條直線，不會多出莫名其妙的 merge commit。因為你們改的是完全不同的檔案（機器人動 data/ 和 docs/latest.json，你動程式碼），實務上不會衝突。

E · 除錯

第一步永遠是展開 log

Actions → 點那次執行 → 左欄點紅色 **build** → 找有紅叉的那一個步驟 → 點三角形展開 → 看最後幾行紅字。「exit code 1」只是「有一步失敗了」的通用訊息，本身沒有資訊量。

症狀	真正的原因	怎麼修
網頁沒更新，但 Actions 有綠勾	綠勾的是 pages build and deployment （每次 commit 自動跑，只重新發佈網站、 <u>不抓資料</u> ），不是 weekly-lab	去左欄點 <code>weekly-lab</code> 手動 Run workflow
網頁沒更新，weekly-lab 也跑完了	瀏覽器快取	Ctrl + Shift + R
資料日期比預期晚一週	stage 選錯（台灣週五選了 fri）	改選 thu 重跑
某指標顯示「待手填」	自動來源抓不到	照流程 B 手填
某指標掛「舊值」標籤	來源超過 10 天沒更新（^MOVE 常態）	持續發生就改走手填備援
Delta 是「一」	只有一筆資料，沒有東西可比	補填前一週的值
板塊「淨資金流」空白	公式需要 兩份快照 （本週規模 - 上週規模 × (1 + 報酬)）	正常。下週跑第二次就會出現
! [rejected] main -> main	遠端有你本機沒有的 commit（機器人推的）	git pull --rebase 然後 git push
cannot pull with rebase: You have unstaged changes	順序錯了，還沒 commit 就 pull	先 git add . 和 git commit，再 pull
Traceback ... ValueError / KeyError / AttributeError	程式本身的 bug	把 Traceback 完整貼給 Claude。這類錯誤本機測不出來，因為沙盒連不到 Yahoo 和 FRED
\$XXX: possibly delisted	某個代號下市或改名	非致命警告。若該列金額長期空白，去 config 換掉那個代號
PowerShell 說找不到路徑	貼上時長路徑被換行截斷	確認提示字元已在 weekly-lab> 底下，那行 cd 可以跳過
Grab 顯示紅色「找不到」	瀏覽器把重複檔名改成 build(1).py	去下載資料夾把括號拿掉改回原檔名

三個「看起來成功但其實沒有」的陷阱

- **Actions 綠勾但資料沒動** —— 綠的是 pages 部署，不是 weekly-lab。認明左欄名稱。
- **PowerShell 整段貼上** —— 前面幾行失敗了後面照跑，錯誤被沖走。一次貼一行。

- **Workflow permissions 沒開 Read and write** —— 排程會正常抓完資料然後 push 失敗，Status 仍可能顯示綠色。檢查 Settings → Actions → General 最下方。

F • 三個檔案的角色

檔案	在哪裡改	改完要不要 Run workflow
data/manual_input.json	GitHub 網頁	要 。那些值要進到指標計算
data/predictions.json	GitHub 網頁	不用 。週二排程會自動處理
config/*.yaml src/*.py docs/index.html	本機 + PowerShell	要 。改了邏輯就要重算

觀察名單增刪走的是流程 B 的變體：在「05 • 設定與手填」點掉或新增個股 → 「產生 YAML」→ 貼回 config/themes.yaml → Run workflow。刪除會立刻反映在 03 分頁（本機套用），新增要等下次抓到資料才有數字。